

PROJECT02 보고서 김현중 2023089507

1. 구현 목표
2. 기존 코드분석
3. 구현 설명
4. 결과
5. 부록

구현 목표

기존의 PCB 코드를 재활용하여 thread를 구현하는 것이다.

system call로 clone과 join을 구현하고 user 코드로 thread_create와 thread_join을 구현하는 것이다.

```
int clone(void(fcn)(void, void*), void *arg1, void *arg2, void *stack);
```

- arg1과 arg2을 매개변수로 받는 fcn을 실행하는 스레드를 만드는 함수이다. stack은 스레드의 스택이다.

```
int join(void **stack);
```

- 자식 스레드가 존재한다면 종료될 때까지 기다리고 종료된 스레드가 있으면 스레드 할당 해제한다.

```
int thread_join();
```

- join system call을 호출하고 stack을 해제한다.

```
int thread_create(void(start_routine)(void, void *), void *arg1, void *arg2);
```

- 스택을 할당하고 clone을 호출한다.

기존 코드분석

구현에 필요한 주요 함수들을 분석해봤다.

1. allocproc
unused process를 찾아서 PCB를 적당히 초기화한다. allocthread를 할 때 참고한다.
2. freeproc
PCB에 할당된 자원들을 해제하고 변수를 초기화한다. freethread를 구현할 때 참고한다.
3. fork
프로세스를 복사할 때 사용한다. clone을 구현할 때 참고한다.
4. wait
자식프로세스가 종료할 때까지 기다리고 종료된 자식 프로세스를 해제한다. join을 구현할 때 참고한다.
5. exit
프로세스를 종료한다. 메인 스레드가 종료하면 스레드 전체를 해제하고 그 외는 호출한 스레드만 종료한다.
6. kill
kill(pid) 호출시 pid만 종료하는 것이 아닌 모든 스레드를 종료한다. 즉 메인스레드를 종료시키는 것과 같다.
7. exec
exec 호출시 다른 모든 스레드를 종료시켜야 한다.
8. proc_pagetable
page할당을 해준다. trapframe_va를 어떻게 매핑시키는 지를 알 수 있다. 구현 설명 PCB추가 사항

```

struct proc{
    ...
    int main_pid;           // Is this the main thread?
    int isThread; // Is this a thread?
    int cnt;                // thread count of the process for mapping
    trapframe_va

    struct proc *main_thread; // Pointer to the main thread of the process
    uint64 stack_base;
};

```

PCB에서 main_pid, isThread, cnt, main_thread, stack_base 변수들을 추가적으로 선언했다. 각각 main thread의 pid, main thread가 갖는 스레드의 개수, main thread로 향하는 포인터, 스레드인지 저장하는 변수, user stack의 주소(나중에 user stack을 free하기 위해 저장)이다. user_stack은 thread일 경우만 사용된다.

스레드 관련 함수들은 paull04_thread.c에 구현했다.

allocthread

```

static struct proc *allocthread(void)
static struct proc*
allocthread(void)
{
    ...
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        p->state = UNUSED;
        release(&p->lock);
        return 0;
    }
    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;
    return p;
}

```

스레드로 사용되지 않는 PCB를 찾아 초기화한다. 상세 설명: proc배열에서 상태가 UNUSED인 PCB를 찾는다. 그리고 PCB를 초기화한다. allocproc와의 큰 차이점은 PCB의 page를 할당하지 않는 것이다. 이 page 할당은 clone에서 다룰 것이다.

paull04_clone

```

int
paull04_clone(uint64 fcn, uint64 arg1, uint64 arg2, uint64 stack)
{
    struct proc *p = myproc();
    struct proc *np;

```

```

uint64 sp;

// Allocate process.
if((np = allocthread()) == 0){
    return -1;
}
p->main_thread->cnt++; // Increment the thread count of the main thread
// Share user memory from parent to child.
np->pagetable = p->pagetable;
np->sz = p->sz;
/*(np->trapframe) = *(p->trapframe);
sp = PGROUNDUP(stack); // Align stack to page size
np->stack_base = stack; // Store the base of the stack for this thread
// Cause fork to return 0 in the child.
*(np->trapframe) = *(p->trapframe); // Copy the parent's trapframe
np->trapframe->sp = sp + PGSIZE; // Set the stack pointer to the top of the new
stack
np->trapframe->ra = 0xFFFFFFFFFFFFFFFF; // Set return address to an invalid
value
np->trapframe->a0 = arg1;
np->trapframe->a1 = arg2;
np->trapframe->epc = fcn; // Set the entry point to the function to be executed
np->trapframe->kernel_sp = np->kstack + PGSIZE; // Set the kernel stack pointer
    //printf("pau1l04_clone: mappages succeeded for trapframe at %lu\n", np-
>trapframe_va);

    np->trapframe_va = TRAPFRAME - (p->main_thread->cnt * PGSIZE); // Set the
virtual address for the trapframe
    if(mappages(p->pagetable, np->trapframe_va, PGSIZE,
                (uint64)(np->trapframe), PTE_R | PTE_W) < 0){
        panic("pau1l04_clone: mappages failed");
        return 0;
    }
    ...
np->state = RUNNABLE;
np->main_pid = p->main_pid; // Set the main thread's PID
np->main_thread = p->main_thread; // Set the main thread pointer
np->isThread = 1; // Mark this as a thread
np->parent = p->main_thread;
int pid = np->pid;
release(&np->lock);
return pid;
}

```

본격적으로 스레드가 이용 가능하게 만드는 과정이다.

중간에 생략된 코드는 file descriptor를 복사하는 건데 명세서에 스레드마다 독립적인 file descriptor를 갖게 하라 했기 때문에 fork처럼 독립적으로 갖게 했다.

스레드는 서로 page를 공유하기에 단순히 부모의 page포인터 값을 스레드의 page포인터 값으로 했다. 유저 스택포인터도 초기화하는데 이를 하기위해서는 stack의 시작 주소를 PGSIZE에 align시켜야 한다. 이는 미리 구현된 매크로를 사용해할 수 있다. 그리고 중요한 부분은 trapframe과 trapframe_va를 초기화하는 부분이다.

trapframe은 allocthread에서 이미 할당됐다. 이를 초기화하는 것은 우선 부모의 trapframe을 복사한다. 그리고 몇몇 부분을 수정하는데 이는 다음과 같다.

```
np->trapframe->sp = sp + PGSIZE; // Set the stack pointer to the top of the new
stack
np->trapframe->ra = 0xFFFFFFFFFFFFFFFF; // Set return address to an invalid
value
np->trapframe->a0 = arg1;
np->trapframe->a1 = arg2;
np->trapframe->epc = fcn; // Set the entry point to the function to be executed
np->trapframe->kernel_sp = np->kstack + PGSIZE; // Set
```

sp는 유저 스택 포인터로 xv6에서 스택포인터는 MAX -> MIN으로 이동해야 하므로 align 시킨 시작 주소에서 PGSIZE를 더한다. ra는 return address로 thread에서는 절대 리턴이 이뤄지면 안되므로 잘못된 리턴값을 뒤서 리턴이 이뤄졌을 때 오류를 일으키게 한다. a0, a1은 함수의 argument로 arg1, arg2로 초기화한다. epc는 PC값으로 fcn을 실행해야 하므로 epc=fcn으로 한다. kernel_sp는 커널 스택포인트로 커널의 시작 주소인 kstack에 PGSIZE를 더한 값을 사용한다.

```
np->trapframe_va = TRAPFRAME - (p->main_thread->cnt * PGSIZE); // Set the
virtual address for the trapframe
if(mappages(p->pagetable, np->trapframe_va, PGSIZE,
    (uint64)(np->trapframe), PTE_R | PTE_W) < 0){
    panic("paull04_clone: mappages failed");
    return 0;
}
```

다음은 trapframe의 가상주소를 할당하는 과정이다. trapframe의 가상주소는 프로세스의 경우 TRAPFRAME 이다. 이 보다 작은 주소 값에는 매핑되지 않으므로 PGSIZE 단위로 겹치지 않게 스레드의 trapframe 가상주소를 할당해 매핑시면 된다. 따라서 main->thread->cnt 값으로 스레드가 몇번 할당됐는지 확인하여 가상주소를 할당한다. 주의할 점은 thread를 해제할 때 절대 cnt값을 감소시키면 안된다. 이후 mappages함수로 trapframe_va에 trapframe을 매핑시킨다.

이후의 코드는 main_thread, parent, main_pid, isThread, state 등을 초기화 한다. thread의 부모는 항상 main thread로 해줬다.

```
void freethread(struct proc *p) {
    //printf("freethread: freeing thread with PID %d\n", p->pid);
    reparent(p);
    if(p->isThread) uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
    p->trapframe_va = 0; // Clear the trapframe virtual address
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
}
```

```

    p->xstate = 0;
    p->state = UNUSED;

    kfree((void *)p->trapframe);
    p->trapframe = 0;
    p->trapframe_va = 0;
    p->main_pid = 0; // Clear the main PID
    p->main_thread = 0; // Clear the main thread pointer
    p->isThread = 0; // Mark this as not a thread anymore
}

```

freethread

thread를 해제하는 함수이다. 처음에 thread를 해제하면 thread의 자식 프로세스들에 대해 parent를 init으로 하 시위해 **reparent(p)** 를 호출한다. 여기서 핵심적인 부분은 **page table** 을 freeproc처럼 해제하지 않는 것이다. page table을 해제하는 건 main thread에 맡기고 page table에 매핑된 **trapframe_va**만 매핑 취소한다. 그 이후 는 freeproc처럼 trapframe을 해제하고 나머지 값들을 0으로 초기화한다.

paul04_join

```

int
paul04_join(uint64 stack){
    struct proc *p = myproc();
    struct proc *np;
    int havekids, pid;

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(np = proc; np < &proc[NPROC]; np++){
            if(np->main_pid == p->main_pid && np->isThread){
                acquire(&np->lock);

                havekids = 1;
                if(np->state == ZOMBIE){
                    if(copyout(p->pagetable, stack, (char *)&np->stack_base, sizeof(uint64))
< 0){
                        release(&np->lock);
                        release(&wait_lock);
                    }
                    //printf("paul04_join: found zombie thread with PID %d\n", np->pid);
                    pid = np->pid;
                    freethread(np);
                    release(&np->lock);
                    release(&wait_lock);
                    return pid;
                }
                release(&np->lock);
            }
        }
    }
}

```

```

    if(!havekids || p->killed){
        release(&wait_lock);
        // printf("paull04_join: no threads to join or process killed\n");
        return -1;
    }
    sleep(p, &wait_lock); //DOC: wait-sleep
}
}

```

자식 thread가 존재하는지 확인하고 존재한다면 thread하나가 종료될 때까지 기다린다. wait과 매우 유사하며 freeproc대신 freethread를 호출한다. 중요한 부분은 **copyout** 함수를 통해 유저 영역에 스택 주소를 보내줘야 한다는 것이다. pagetable에서 **stack** 변수에 해당하는 부분에 스택 시작 주소를 전달한다.

syscall wrappers

paull04_join과 **paull04_clone**은 syscall 등록을 위한 wrapper로 sys_join, sys_clone을 갖는다.

thread_create

```

int
thread_create(void(*start_routine)(void*, void*), void *arg1, void *arg2)
{
    void *stack = malloc(PGSIZE << 1);
    if (stack == 0) {
        return -1;
    }

    int pid = clone(start_routine, arg1, arg2, stack);
    if (pid < 0) {
        free(stack);
        return -1;
    }

    return pid;
}

```

stack을 malloc으로 할당해주고 clone system call을 호출하여 스레드를 생성한다. 실패시에는 -1을 리턴하고 성공하면 pid를 리턴한다.

```

int
thread_join()
{
    void *stack = 0;
    int pid = join(&stack);
    if(stack) free(stack);
    if(pid < 0) {
        return -1; // Error in join
    }
}

```

```
    return pid; // Return the thread's PID
}
```

thread_join

join을 호출하여 thread가 종료될 때까지 기다린다. stack에 대한 포인터 주소를 join에 매개변수로 보냄으로써 스택 시작 부분 주소를 얻고 이를 free한다.

그 외 수정된 함수 리스트

fork, userinit, wait, freeproc, kill, exit, exec, growproc

fork, userinit

```
np->isThread = 0; // Not a thread
np->main_pid = pid;
np->main_thread = np;
np->cnt = 0;
```

새로 PCB에 추가한 변수들을 초기화하는 코드를 추가했다. 이는 다음과 같다. clone과는 다르게 프로세스를 생성하는 것이기에 main_pid, main_thread는 자기 자신으로 하고, 나머지는 0으로 한다.

wait

```
int
wait(uint64 addr){
...
    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p && pp->isThread == 0) {
                ...
            }
            ...
        }
        ...
    }
    ...
}
```

wait 함수에서 변경된 부분은 자식 프로세스가 thread인지 체크하는 부분이다. thread는 자식 프로세스로 인식하지 않게 했다.

freeproc

kill

```

int
kill(int pid)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            release(&p->lock);
            acquire(&p->main_thread->lock);
            p = p->main_thread;
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}

```

수정한 부분은 **p->killed** 를 1로 하는 것이 아닌, p의 main_thread의 killed를 1로 하는 것이다. 이는 exit함수와 관련이 있다. 명세서에서는 한 스레드가 kill되면 모든 스레드를 종료시켜야 한다고 했다. 메인 함수가 exit하면 모든 스레드를 해제하도록 구현했다. 따라서 main_thread의 killed를 1로하면 main thread가 cpu를 잡았을 때 exit함수를 호출하고 exit에서 모든 스레드를 종료하게 했다.

exit

```

void
exit(int status)
{
    struct proc *p = myproc();
    if(p == initproc)
        panic("init exiting");

    if(!p->isThread) cleanAllThreads(p);

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){
            struct file *f = p->ofile[fd];
            fileclose(f);
            p->ofile[fd] = 0;
        }
    }

    begin_op();
}

```

```

    iput(p->cwd);
    end_op();
    p->cwd = 0;

    acquire(&wait_lock);

    // Give any children to init.
    reparent(p);

    // Parent might be sleeping in wait().
    wakeup(p->parent);

    acquire(&p->lock);

    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}

```

exit에서는 우선 exit한 스레드가 main thread인지 아닌지 구분한다. 메인스레드가 호출한 경우면 cleanAllThreads를 호출하여 모든 자식 스레드를 해제해준다. 이 부분을 원래 exit 구현에서 추가했다.

exec

```

int
exec(char *path, char **argv)
{
    char *s, *last;
    int i, off;
    uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
    struct elfhdr elf;
    struct inode *ip;
    struct proghdr ph;
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();
    if(p -> isThread) uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
    ...
    // Commit to the user image.
    int mp = p->main_pid; // Save the main process's pid
    struct proc *mt = p->main_thread; // Save the main thread pointer
    p->main_pid = p->pid; // Set the main thread's pid
    p->main_thread = p; // Set the main thread pointer to itself
    p->parent = p->main_thread->parent;
    p->isThread = 0; // Not a thread
    p->cnt = 0; // Initialize thread count
}

```

```

    cleanAllThreads(mt);
    if(p->pid == mp){
        proc_freepagetable(oldpagetable, oldsz);
    }
    else {
        mt->killed=1;
    }

    return argc; // this ends up in a0, the first argument to main(argc, argv)
    ...
}

```

위는 수정한 exec에서 중요한 부분이다. 먼저 exec을 호출한 스레드가 main thread인지 확인한다. main thread가 아니면 trapframe의 가상 주소를 page table에서 매핑을 해제한다. 이는 나중에 page table을 free할 때 매핑된 주소들이 있으면 오류가 발생하기에 반드시 해야한다. 그 아래는 PCB변수들을 초기화는 부분이다. exec은 page를 새로 할당하는데 이전에 사용된 page는 반드시 해제되어야 한다. 만약에 exec을 호출한 스레드가 메인 스레드였으면 page table을 해제하고 아니라면 메인 스레드에 killed를 1로 설정하여 메인스레드에서 해제하게 한다. 그리고 둘다 자식스레드들을 해제해준다.

결과

```

[TEST#1]
Thread 0 start
Thread 1 start
Thread 1 end
Thread 2 start
Thread 2 end
Thread 3 start
Thread 3 end
Thread 4 start
Thread 4 end
Thread 0 end
TEST#1 Passed

[TEST#2]
Thread 0 start, iter=0
Thread 0 end
Thread 1 start, iter=1000
Thread 1 end
Thread 2 start, iter=2000
Thread 2 end
Thread 3 start, iter=3000
Thread 3 end
Thread 4 start, iter=4000
Thread 4 end
TEST#2 Passed

[TEST#3]
Thread 0 start
Thread 1 start
Thread 2 start

```

```
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#3 Passed
```

```
[TEST#4]
Thread 0 sbrk: old break = 0x00000000000015000
Thread 0 sbrk: increased break by 14000
new break = 0x00000000000029010
Thread 1 size = 0x00000000000029010
Thread 2 size = 0x00000000000029010
Thread 3 size = 0x00000000000029010
Thread 4 size = 0x00000000000029010
Thread 0 sbrk: free memory
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#4 Passed
```

```
[TEST#5]
Thread 0 start, pid 29
Thread 1 start, pid 29
Thread 2 start, pid 29
Thread 3 start, pid 29
Thread 4 start, pid 29
Thread 0 end
TEST#5 Passed
```

```
[TEST#6]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Thread exec test 0
TEST#6 Passed
```

```
All tests passed. Great job!!
```

모든 주어진 테스트를 통과한 것을 보아 올바르게 구현한 것을 알 수 있다.

issue

trapframe_va를 어떻게 할당해야 하는지 문제가 있었다. 이는 TRAPFRAME 이전의 주소에 할당하면 된다는 것을 알 수 있었고 TRAPFRAME - PGSIZE*스레드개수로 할당할 수 있었다.

그리고 exec에서 freewalk 오류가 계속발생했었는데 이는 exec의 스레드를 호출한 trapframe_va를 매핑해제를 하지 않아 발생함을 깨닫고 스레드인지 체크하여 이를 해결했다.